GUÍA PRÁCTICA · SEMANA 2

JavaFX + Scene Builder + FXML

Construcción paso a paso de un sistema visual de gestión de maquinaria

Asignatura: Desarrollo Orientado a Objetos (Java)

Modalidad: taller guiado. Realice cada paso, guarde y compruebe antes de avanzar.

Entorno: IntelliJ IDEA, JDK 21, Maven, JavaFX 21 y Scene Builder. Los nombres de menús pueden variar según la versión.

Resultado: ventana principal con tabla y ventana modal de registro; botones de apertura/cierre y campos que cambian según el tipo seleccionado.

Alcance: esta semana NO implementaremos clases de dominio, registros reales, validaciones ni persistencia Gson/JSON.

Ruta de aprendizaje

Bloque	Pasos	Producto
A. Preparación	1–6	Proyecto Maven y dependencias JavaFX
B. Ventana principal	7–20	principal.fxml con BorderPane y TableView
C. Formulario	21–23	registro.fxml en Scene Builder
D. Navegación	24–28	Stage secundario modal
E. Eventos	29–36	Cancelar, ComboBox y campos dinámicos

A. Preparación del proyecto (pasos 1 al 6)

Paso 1. Crear un proyecto JavaFX con Maven

1. Abra IntelliJ IDEA → New Project. Seleccione una plantilla JavaFX con sistema de construcción Maven (si su versión ofrece el asistente).

2. Nombre del proyecto: GestionMaquinariaSceneBuilder. Utilice JDK 21.

3. Configure el módulo Java como cl.duoc.gestionmaquinaria y cree el paquete cl.duoc.gestionmaquinaria.

4. Espere a que Maven descargue las dependencias. Si el asistente creó clases de ejemplo, no es necesario borrarlas ahora.

Nota: si la plantilla genera un nombre de paquete distinto, adapte las rutas y el archivo pom.xml de forma consistente. En esta guía trabajaremos con el paquete indicado.

Paso 2. Verificar las dependencias Maven

En pom.xml, compruebe que están las dependencias javafx-controls y javafx-fxml con versión 21, además del plugin javafx-maven-plugin. Si su plantilla ya las incorpora, no las duplique. Fragmento de referencia:

```
<properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <javafx.version>21</javafx.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.openjfx</groupId>
        <artifactId>javafx-controls</artifactId>
        <version>${javafx.version}</version>
    </dependency>
    <dependency>
        <groupId>org.openjfx</groupId>
        <artifactId>javafx-fxml</artifactId>
        <version>${javafx.version}</version>
    </dependency>
</dependencies>
```

En la configuración del plugin, la clase principal debe ser cl.duoc.gestionmaquinaria/cl.duoc.gestionmaquinaria.App. El plugin puede venir configurado desde la plantilla.

Paso 3. Preparar carpetas

```
src/main/java/
  module-info.java
  cl/duoc/gestionmaquinaria/App.java
  cl/duoc/gestionmaquinaria/controller/
src/main/resources/
  cl/duoc/gestionmaquinaria/principal.fxml
```

1. En src/main/java, cree el paquete cl.duoc.gestionmaquinaria.controller (quedará vacío por ahora).

2. En src/main/resources, cree las carpetas cl/duoc/gestionmaquinaria.

3. Más adelante guardaremos ahí ambos archivos FXML.

Paso 4. Crear module-info.java

```
module cl.duoc.gestionmaquinaria {
    requires javafx.controls;
    requires javafx.fxml;
    opens cl.duoc.gestionmaquinaria to javafx.fxml;
    exports cl.duoc.gestionmaquinaria;
}
```

Importante: todavía NO agregue la línea opens ...controller porque ese paquete no contiene ninguna clase compilada. La añadiremos al crear el primer controlador.

Paso 5. Crear App.java

```
package cl.duoc.gestionmaquinaria;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;
import java.io.IOException;

public class App extends Application {
    @Override
    public void init() {
        System.out.println("1. init(): preparando aplicacion");
    }

    @Override
    public void start(Stage stage) throws IOException {
        System.out.println("2. start(): creando ventana");
        FXMLLoader loader = new FXMLLoader(
                App.class.getResource("principal.fxml"));
        Scene scene = new Scene(loader.load(), 850, 550);
        stage.setTitle("Sistema de Gestion de Maquinaria");
        stage.setScene(scene);
        stage.show();
    }

    @Override
    public void stop() {
        System.out.println("3. stop(): cerrando aplicacion");
    }

    public static void main(String[] args) {
        System.out.println("0. main(): iniciando JavaFX");
        launch(args);
    }
}
```

Paso 6. Comprender Stage, Scene y ciclo de vida

Elemento	Qué representa
Stage	La ventana del sistema operativo.
Scene	El contenido visual que se coloca dentro de la ventana.
FXMLLoader	Lee un archivo .fxml y construye sus controles.
main() / launch()	Inicia JavaFX.
init() / start() / stop()	Preparación / presentación / cierre de la aplicación.

Atención: App.java intentará abrir principal.fxml. Primero diseñaremos y guardaremos ese archivo; después ejecutaremos la aplicación.

B. Diseñar principal.fxml (pasos 7 al 20)

Paso 7. Crear la raíz BorderPane

1. Abra Scene Builder → File → New. En Library busque BorderPane y úselo como contenedor raíz.

2. Seleccione el BorderPane en Hierarchy y configure Pref Width = 850 y Pref Height = 550.

3. Guarde con File → Save As en src/main/resources/cl/duoc/gestionmaquinaria/principal.fxml.

Concepto: BorderPane distribuye el contenido en TOP, CENTER, BOTTOM, LEFT y RIGHT.

Paso 8. Agregar un VBox en TOP

1. Busque VBox en Library y arrástrelo a la región TOP del BorderPane.

2. En Inspector → Layout configure Spacing = 15, Alignment = CENTER_LEFT y Padding = 20 en los cuatro lados.

Paso 9. Agregar el título

1. Arrastre un Label dentro del VBox.

2. En Inspector → Properties → Text escriba Sistema de Gestión de Maquinaria.

3. Configure el tamaño de fuente en 22.

Paso 10. Agregar la barra horizontal

1. Dentro del VBox, debajo del título, arrastre un HBox.

2. Configure Spacing = 10 y Alignment = CENTER_LEFT.

Paso 11. Agregar el buscador

1. Arrastre un TextField dentro del HBox.

2. En Inspector → Code configure fx:id = txtBuscar.

3. En Properties configure Prompt Text = Buscar por código..., Text vacío y Pref Width = 300.

Nota: el texto de ayuda puede ocultarse cuando el TextField tiene el foco. Haga clic en otro control para verlo; no es un error.

Paso 12. Agregar el botón de registro

1. Arrastre un Button a la derecha del buscador.

2. En Properties → Text escriba Agregar maquinaria.

3. En Code → fx:id escriba btnAgregar.

4. Deje On Action vacío por ahora.

Paso 13. Guardar el diseño inicial

1. Guarde con Ctrl + S.

2. Verifique en Hierarchy: BorderPane → TOP VBox → Label y HBox → TextField y Button.

Paso 14. Ejecutar la primera ventana

1. En IntelliJ ejecute Maven → Plugins → javafx → javafx:run.

2. Compruebe que aparece el título, el buscador y el botón. El resto de la ventana aún estará vacío.

3. Revise los mensajes de main, init y start en la consola.

Comprobación: La ventana abre sin excepciones y tiene tamaño aproximado de 850 × 550.

Pasos 15 y 16. Verificar el Prompt Text

1. Si aparece el cursor dentro del buscador, haga clic en el título o en el botón para quitarle el foco.

2. Confirme que el TextField tiene Text vacío y que la ayuda está escrita en Prompt Text, no en Text.

Paso 17. Agregar la TableView en CENTER

1. Abra principal.fxml en Scene Builder y arrastre TableView a la región CENTER del BorderPane.

2. Seleccione la TableView y configure fx:id = tablaMaquinas en Inspector → Code.

3. En Hierarchy despliegue TableView; renombre las columnas existentes y agregue las faltantes arrastrando TableColumn.

4. Configure las cinco columnas con los textos y fx:id de la tabla siguiente.

Text visible	fx:id	Ancho aprox.
Tipo	colTipo	160
Código	colCodigo	120
Horas de uso	colHoras	140
Potencia	colPotencia	140
Costo servicio	colCosto	170

Si Inspector ofrece BorderPane Margin para la tabla, configure Top 10, Right 20, Bottom 10, Left 20. La tabla estará vacía: aún no existe ningún modelo de datos.

Paso 18. Guardar y ejecutar la tabla

1. Guarde principal.fxml.

2. Ejecute javafx:run y compruebe que la TableView aparece con cinco encabezados.

3. Es normal que muestre No content in table o un mensaje similar.

Paso 19. Diseñar BOTTOM

1. En Scene Builder arrastre un HBox a la región BOTTOM.

2. Configure Spacing = 15, Alignment = CENTER_LEFT y Padding = 15.

3. Agregue un Button: Text = Eliminar seleccionado, fx:id = btnEliminar. Deje On Action vacío.

4. Agregue un Label: Text = Sistema listo. Esperando acciones del usuario., fx:id = lblEstado.

Paso 20. Probar la ventana principal completa

1. Guarde y ejecute javafx:run.

2. Confirme que TOP contiene título y barra de herramientas; CENTER, la tabla; BOTTOM, botón Eliminar y etiqueta de estado.

Comprobación: Todavía no deben funcionar Buscar, Agregar o Eliminar; Agregar se conectará en el bloque D.

C. Diseñar registro.fxml (pasos 21 al 23)

Paso 21. Crear la segunda interfaz

1. En Scene Builder use File → New y coloque un VBox como contenedor raíz.

2. Configure Pref Width = 500, Pref Height = 650, Spacing = 10, Padding = 20 y Alignment = TOP_LEFT.

3. Agregue los controles en el orden indicado en las tablas siguientes. Cada Label va antes de su campo.

Tipo de control	Text / Prompt Text	fx:id
Label	Nueva maquinaria	(sin fx:id)
Label	Tipo de maquinaria	(sin fx:id)
ComboBox	Inicialmente vacío	cmbTipo
Label	Código	(sin fx:id)
TextField	Ejemplo: EX001	txtCodigo
Label	Horas de uso	(sin fx:id)
TextField	Ejemplo: 0	txtHoras
Label	Potencia	(sin fx:id)
TextField	Ejemplo: 250	txtPotencia

Tipo de control	Text visible	fx:id
Label	Peso de la excavadora	lblPeso
TextField	Prompt: Peso en toneladas	txtPeso
CheckBox	Mantención al día	chkMantencion
CheckBox	Certificación vigente	chkCertificacion

Tipo de control	Text visible	fx:id
Label	Capacidad del balde (m³)	lblCapacidadBalde
TextField	Prompt: Capacidad en m³	txtCapacidadBalde
HBox	Spacing = 10	(sin fx:id)
Button dentro del HBox	Cancelar	btnCancelar
Button dentro del HBox	Guardar maquinaria	btnGuardar

Importante: al inicio todos los campos estarán visibles. No configure On Action todavía. Los campos se mostrarán u ocultarán en el paso 35.

Paso 22. Guardar registro.fxml

1. Elija File → Save As.

2. Guarde con el nombre registro.fxml en la misma carpeta de principal.fxml: src/main/resources/cl/duoc/gestionmaquinaria/.

Paso 23. Vista previa del formulario

1. En Scene Builder use Preview → Show Preview in Window.

2. Revise que los controles estén en orden y que los botones se vean al final.

3. Si faltan controles por altura, reduzca el Spacing o aumente el alto de la ventana de diseño; no elimine campos.

Comprobación: Ahora existen principal.fxml y registro.fxml; el segundo todavía no se abre desde la aplicación.

D. Abrir una segunda ventana (pasos 24 al 28)

Paso 24. Entender la navegación

Al presionar Agregar maquinaria en la ventana principal, un controlador cargará registro.fxml mediante FXMLLoader, creará una nueva Scene y la instalará en un nuevo Stage. El Stage será modal: mientras esté abierto, bloqueará la interacción con las otras ventanas de la aplicación.

Paso 25. Crear PrincipalController.java

En IntelliJ cree una clase Java llamada PrincipalController dentro del paquete cl.duoc.gestionmaquinaria.controller. Copie el archivo completo:

```
package cl.duoc.gestionmaquinaria.controller;

import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Modality;
import javafx.stage.Stage;
import java.io.IOException;

public class PrincipalController {

    @FXML
```

```
    private void abrirRegistro() {
        try {
            FXMLLoader loader = new FXMLLoader(
                getClass().getResource(
                    "/cl/duoc/gestionmaquinaria/registro.fxml")
            );

            Scene scene = new Scene(loader.load());
            Stage ventanaRegistro = new Stage();
            ventanaRegistro.setTitle("Registro de maquinaria");
            ventanaRegistro.setScene(scene);
            ventanaRegistro.initModality(Modality.APPLICATION_MODAL);
            ventanaRegistro.showAndWait();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Explique: @FXML habilita el método para enlazarlo desde el archivo FXML; new Stage() crea una ventana; showAndWait() la muestra y espera a que se cierre.

Paso 26. Asociar PrincipalController en Scene Builder

1. Abra principal.fxml en Scene Builder.

2. En Document → Controller → Controller class escriba cl.duoc.gestionmaquinaria.controller.PrincipalController.

3. Seleccione el botón Agregar maquinaria. En Inspector → Code → On Action escriba abrirRegistro.

4. Guarde con Ctrl + S.

En el archivo FXML el evento quedará asociado al método del controlador (Scene Builder suele escribir onAction="#abrirRegistro").

Paso 27. Actualizar module-info.java

Ahora que existe una clase en el paquete controller, reemplace module-info.java por:

```
module cl.duoc.gestionmaquinaria {
    requires javafx.controls;
    requires javafx.fxml;

    opens cl.duoc.gestionmaquinaria to javafx.fxml;
    opens cl.duoc.gestionmaquinaria.controller to javafx.fxml;

    exports cl.duoc.gestionmaquinaria;
}
```

Paso 28. Ejecutar y probar la navegación

1. En IntelliJ, guarde todos los archivos. Si lo necesita, ejecute Maven → Lifecycle → clean.

2. Ejecute javafx:run.

3. Presione Agregar maquinaria: debe abrirse el formulario registro.fxml.

4. Intente hacer clic en la ventana principal mientras la secundaria está abierta: la modalidad impedirá interactuar con ella.

5. Cierre la ventana secundaria con la X y confirme que la principal sigue abierta.

Comprobación: Si el controlador no se encuentra, compruebe la ruta del paquete, fx:controller y la línea opens en module-info.java.

E. Eventos y controles dinámicos (pasos 29 al 36)

Paso 29. Crear RegistroController.java: botón Cancelar

Cree RegistroController.java en el mismo paquete controller. Primera versión:

```
package cl.duoc.gestionmaquinaria.controller;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.stage.Stage;

public class RegistroController {

    @FXML
    private Button btnCancelar;

    @FXML
    private void cancelar() {
        Stage ventana = (Stage)
                btnCancelar.getScene().getWindow();
        ventana.close();
    }
}
```

El botón obtiene su Scene y, a través de ella, el Stage que debe cerrarse. No cierra la aplicación completa.

Paso 30. Conectar Cancelar desde Scene Builder

1. Abra registro.fxml.

2. En Document → Controller configure cl.duoc.gestionmaquinaria.controller.RegistroController.

3. Seleccione Cancelar y verifique fx:id = btnCancelar y On Action = cancelar.

4. Guarde.

Paso 31. Probar Cancelar

1. Ejecute la aplicación. Abra el formulario con Agregar maquinaria.

2. Pulse Cancelar y compruebe que solo se cierra la ventana secundaria.

Paso 32. Agregar opciones al ComboBox

Reemplace temporalmente el contenido de RegistroController.java por esta segunda versión:

```
package cl.duoc.gestionmaquinaria.controller;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.stage.Stage;

public class RegistroController {
    @FXML private Button btnCancelar;
    @FXML private ComboBox<String> cmbTipo;

    @FXML
    private void initialize() {
        cmbTipo.getItems().addAll("Excavadora", "Cargador Frontal");
```

```
    }

    @FXML
    private void cancelar() {
        Stage ventana = (Stage)
                btnCancelar.getScene().getWindow();
        ventana.close();
    }
}
```

Concepto: FXMLLoader ejecuta initialize() una vez que ha cargado los controles e inyectado los campos @FXML. No es el mismo método que App.init().

Paso 33. Revisar el fx:id del ComboBox

1. En registro.fxml seleccione el ComboBox.

2. Verifique en Inspector → Code que su fx:id sea cmbTipo.

3. Guarde. No agregue manualmente las opciones desde Scene Builder.

Paso 34. Probar el ComboBox

1. Ejecute la aplicación y abra Registro de maquinaria.

2. Despliegue el ComboBox y compruebe que aparecen Excavadora y Cargador Frontal.

3. Seleccione cada opción; todavía no deben ocultarse los campos.

Paso 35. Cambiar campos según el tipo seleccionado

Primero, en Scene Builder verifique estos fx:id: cmbTipo, lblPeso, txtPeso, chkMantencion, chkCertificacion, lblCapacidadBalde y txtCapacidadBalde. Después reemplace el archivo RegistroController.java completo por esta versión final:

```
package cl.duoc.gestionmaquinaria.controller;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.stage.Stage;

public class RegistroController {

    @FXML private Button btnCancelar;
    @FXML private ComboBox<String> cmbTipo;
    @FXML private Label lblPeso;
    @FXML private TextField txtPeso;
    @FXML private CheckBox chkMantencion;
    @FXML private CheckBox chkCertificacion;
    @FXML private Label lblCapacidadBalde;
    @FXML private TextField txtCapacidadBalde;

    @FXML
    private void initialize() {
        cmbTipo.getItems().addAll(
                "Excavadora", "Cargador Frontal");
        cambiarTipo();
    }

    @FXML
    private void cambiarTipo() {
        String tipo = cmbTipo.getValue();
        boolean esExcavadora = "Excavadora".equals(tipo);
```

```
        boolean esCargador = "Cargador Frontal".equals(tipo);

        lblPeso.setVisible(esExcavadora);
        lblPeso.setManaged(esExcavadora);
        txtPeso.setVisible(esExcavadora);
        txtPeso.setManaged(esExcavadora);
        chkMantencion.setVisible(esExcavadora);
        chkMantencion.setManaged(esExcavadora);
        chkCertificacion.setVisible(esExcavadora);
        chkCertificacion.setManaged(esExcavadora);

        lblCapacidadBalde.setVisible(esCargador);
        lblCapacidadBalde.setManaged(esCargador);
        txtCapacidadBalde.setVisible(esCargador);
        txtCapacidadBalde.setManaged(esCargador);
    }

    @FXML
    private void cancelar() {
        Stage ventana = (Stage)
                btnCancelar.getScene().getWindow();
        ventana.close();
    }
}
```

En Scene Builder, seleccione el ComboBox y configure Inspector → Code → On Action = cambiarTipo. Guarde.

¿Por qué dos métodos? setVisible(false) oculta el control, pero el VBox podría reservar su espacio. setManaged(false) permite que el contenedor lo excluya del cálculo del diseño. Usamos ambos para que el formulario se reorganice.

Paso 36. Prueba funcional final

☐ La ventana principal abre sin errores.

☐ Agregar maquinaria abre el formulario modal.

☐ Al iniciar, los campos exclusivos están ocultos.

☐ Excavadora muestra Peso, Mantención y Certificación.

☐ Cargador Frontal muestra Capacidad del balde.

☐ Al cambiar de tipo no quedan espacios vacíos innecesarios.

☐ Cancelar cierra el formulario y permite volver a abrirlo.

F. Estructura final y resolución de problemas

```
GestionMaquinariaSceneBuilder/
  pom.xml
  src/main/java/
    module-info.java
    cl/duoc/gestionmaquinaria/
      App.java
      controller/
        PrincipalController.java
        RegistroController.java
  src/main/resources/
    cl/duoc/gestionmaquinaria/
      principal.fxml
      registro.fxml
```

Problema	Qué revisar
Location is not set / FXML no encontrado	Ruta y nombre de principal.fxml o registro.fxml; mayúsculas/minúsculas.
Package ...controller not found	No abrir un paquete inexistente en module-info; crear controlador primero.
Error al ejecutar evento	Coincidencia exacta entre On Action y método @FXML.
NullPointerException en cmbTipo	fx:id del ComboBox y fx:controller de registro.fxml.
Los campos no desaparecen bien	Usar setVisible y setManaged para cada control correspondiente.
El texto de ayuda no aparece	Text vacío; Prompt Text correcto; cambiar el foco.
Advertencia de versión JavaFX 26/21	Scene Builder puede guardar un namespace más nuevo; usar versiones compatibles si surgen problemas reales.

G. Entrega y cierre de la Semana 2

Entregable sugerido: proyecto Maven comprimido, con archivos Java, ambos FXML y pom.xml. Antes de comprimir, comprobar que se ejecuta desde IntelliJ con javafx:run.

Preguntas de reflexión: ¿En qué se diferencia Stage de Scene? ¿Para qué sirve FXMLLoader? ¿Por qué usamos fx:id? ¿Qué hace initialize()? ¿Por qué conviene usar setManaged junto con setVisible()?

Próximas semanas (3 y 4, juntas): modelo orientado a objetos, arquitectura MVC, conexión real con TableView, validaciones y persistencia JSON con Gson. Estas funcionalidades no forman parte de la Semana 2.

Versión de respaldo recomendada:

```
git add .
git commit -m "Semana 2 - Interfaces JavaFX con Scene Builder y FXML"
```